Prove Less, Repair Better: Automation-Friendly Program Repair for Small Models in Lean 4

Course: Agentic Software Engineering (AutoSE, CS6.403) | Team: Agents of Ultron

Members: Inesh Dheer (2023111010) · Poojitha J (2024101088) · Gracy Garg (2023101118)

Abstract

We test whether small, locally runnable coding models produce more verified software repairs when they repair the implementation into a form that a fixed Lean 4 automation suite can prove, instead of writing proofs themselves. Small models are weak at Lean proof generation, and even frontier models rarely produce valid proofs on verified-code benchmarks such as VERINA. We therefore shift the proof burden from the model to the shape of the code. We compare this automation-friendly repair against single-shot generation and feedback-guided proof-writing, using 1.5B and 3B models on an 8 GB GPU, matched compute budgets, immutable contracts, and hidden behavioural tests. The primary outcome is the verified repair rate: patches that satisfy the unchanged formal contract and pass independent behavioural checks.

1. Background and Research Gap

Proof generation is the bottleneck in verified code generation, and it is worst for small models. On VERINA (189 Lean 4 tasks), the best model in the published evaluation reached 72.6% code correctness but only 4.9% proof success, with one trial per task [1]. Small, locally runnable models are weaker still, yet they are what developers can run privately and cheaply on a laptop.

Lean 4 lets an implementation be checked against a machine-checked contract. A passing proof establishes the stated property, subject to the contract being correct. Lean also ships strong automation (`simp` , `omega` , `decide` , `aesop` , `grind`) that can discharge many obligations without hand-written tactics, provided the code has a suitable structure.

Existing work attacks the proof side or uses large models.

- **Compiler-guided proof repair.** APRIL trains models to repair Lean proofs from compiler errors and goal states [2]. APOLLO combines Lean diagnostics, decomposition and automated solvers for theorem proving [3].

- **Proof refactoring.** ImProver and Lean Refactor rewrite or compress existing proofs [4, 5].
- **Joint planning and repository scale.** P³ plans programs and proofs jointly [6]; Vero evaluates frontier agents building verified repositories [7].
- **Code restructuring for verification.** Dissect-and-Restore transiently refactors Dafny code to help LLMs verify it, relying on SMT automation [8]. DafnyFix repairs faulty Dafny implementations with deterministic single transformations [15]. Neither uses small language models or Lean 4 automation.
- **Verifier scaffolds.** An MIT thesis reports a Lean verifier-guided search scaffold and documents specification hacking under verifier rewards [9].
- **Feedback format.** FeedbackEval, property-oriented feedback, and an ASE 2026 study compare feedback types for LLM repair [10, 11, 12].

Research gap. No study we know of tests whether small models do better at verified software repair by changing the *implementation* to suit a *fixed, pre-registered* Lean automation suite, rather than by writing proofs. This reallocates reasoning: the model does what small models do comparatively well (write code), and deterministic automation does the proof.

We do not claim novelty for compiler feedback, structured repair messages, automated tactics, or code restructuring in isolation. The contribution is the combination: permanent, behaviour-preserving implementation repair in Lean 4; automation-only proof discharge with no model-written tactics; laptop-scale models; and a direct, budget-matched comparison against proof-writing repair.

Proposed contributions

1. An automation-friendly repair method for small models in Lean 4: the model edits code only, and a fixed tactic portfolio proves it.
2. A controlled, budget-matched comparison against single-shot generation and feedback-guided proof-writing at 1.5B and 3B scale.
3. An analysis of which defect types benefit, how often zero model-written tactics suffice, and what the method costs in code quality, runtime and integrity risk.
4. A reproducible local harness: repair tasks derived from VERINA, evaluation scripts, logs and the pre-registered tactic portfolio.

2. Research Questions and Hypotheses

RQ1 — Effectiveness (primary). At matched compute, does automation-friendly repair achieve a higher verified repair rate than feedback-guided proof-writing for 1.5B and 3B models?

H1: Automation-friendly repair yields a higher verified repair rate than proof-writing, because small models generate suitable implementations more reliably than proofs. We will report effect sizes with uncertainty, including task categories where the method gives no benefit or does worse.

RQ2 — Mechanism. What share of successful repairs needs zero model-written tactics, and which defect types benefit most?

H2: Local defects (boundary conditions, wrong operators, empty-input handling) benefit most. Defects that require a different algorithm benefit least. We treat this as a hypothesis to test, not an expected result.

RQ3 — Engineering trade-offs. Does code-shaping preserve code quality and intended behaviour? What does it cost in latency, model calls, patch size, regressions and integrity violations?

H3: Code-shaping may produce larger or less natural patches, and may tempt the model to copy an executable specification into the implementation. We will quantify these costs and report whether the gain in verified repairs justifies them.

Secondary (optional, only if time allows). Does progressive hinting (minimal goal first, then the spec's structure and relevant declarations) improve code-shaping over minimal hints alone, and does the effect differ between 1.5B and 3B?

3. Proposed Method and Experimental Design

The model repairs only the implementation; a fixed automation suite must prove the unchanged contract with no model-written tactics.

Task input

Each task provides a faulty Lean 4 implementation, a natural-language requirement, a fixed formal contract, visible tests, and an executable verification environment. Hidden behavioural tests stay separate from agent-visible inputs. The contract, hidden tests and verification configuration are immutable in every condition.

Automation-friendly repair loop

1. **Preflight.** Compile, run visible tests, and run the tactic portfolio on the faulty code to record the starting state.
2. **Sample.** The small model generates k = 4 candidate implementations per round. The prompt contains the requirement, contract, faulty code, failing test or counterexample, and the unclosed proof goal. It asks for code only, never tactics.
3. **Filter.** Discard candidates that fail compilation, visible tests or property checks.

4. **Prove by automation only.** Run the pre-registered tactic portfolio on each survivor. The first candidate the portfolio closes is accepted.
5. **Shape the next round.** If no candidate closes, return the unclosed goal and a structural hint (for example, the recursion pattern of the contract) and ask for an implementation whose structure mirrors the specification.
6. **Stop.** Stop on an accepted patch or when the fixed budget is exhausted, reporting the task as unresolved.

Pre-registered tactic portfolio

The portfolio is fixed before the pilot and reported in full: `simp`, `omega`, `decide`, `aesop`, `grind`, plus a small documented set of induction templates (structural induction on the main argument, each case closed by the same tactics). Tactics run in a fixed order, each with a per-attempt timeout, against pinned Lean and Mathlib versions. Any change after the pilot is reported as a deviation, with its reason, and results from different portfolios are never pooled. A failed automated proof is not treated as evidence of a wrong implementation: the candidate may be wrong, the property may need stronger reasoning, or the portfolio may lack the budget. This prevents comparing a fixed baseline against a progressively hand-tuned system.

Conditions

All conditions use the same models and faulty implementations. Condition A is a single-shot lower bound. The principal comparison, B versus C, shares the same maximum model calls, total token budget, wall-clock limit, initial task and symbolic preflight, so any advantage of C cannot come from extra attempts. We report model usage, Lean CPU time and total runtime separately.

Condition	What the model does	Feedback	Purpose
A. Single-shot generation	Writes implementation and proof once	None	Lower-bound baseline
B. Feedback-guided proof-writing	Iteratively repairs implementation and proof	Lean errors, goals, tests; same preflight as C	Fair control: isolates code-shaping
C. Automation-friendly repair (proposed)	Writes implementation only; portfolio proves it	Unclosed goals, tests, structural hints	The contribution
D. C + progressive hints (optional)	As C; richer hints only after a failed round	Escalating structural context	Secondary experiment

The paper's main comparison is B versus C. A versus C shows the gain over naive use; D is run only if the main study finishes early.

Models

Qwen2.5-Coder 1.5B and 3B, quantized to 4-bit, with contexts of about 4K tokens. A 7B model is an optional reference only if peak VRAM measurements allow. No fine-tuning or reinforcement learning in this study.

Benchmark

Primary: VERINA [1], starting with its basic tier. We build repair tasks by injecting validated defects into reference implementations: wrong boundary conditions, wrong operators, flawed recursive cases, and mishandled empty inputs. A defect is kept only if it fails hidden tests or the contract and the reference passes both. Variants of one task stay in the same split. CLEVER [13] is an optional external replication, not a prerequisite.

Integrity rules

- No `sorry`, `admit`, or new axioms in any accepted patch.
- No edits to the contract, hidden tests or verification configuration. In condition C the code-only rule is enforced mechanically: the model can write only the designated implementation region, while the contract, theorem statement, proof harness and hidden tests sit outside it. Any candidate that adds proof text is rejected.
- Suspected hard-coded outputs, tautological implementations and specification-copying shortcuts are recorded and investigated. Structural similarity to the specification alone is not a violation if the implementation is legitimate and meets the requirements. CLEVER's non-computable specifications block this by design; on VERINA we add complexity and runtime checks.

Assessment outcomes

We keep these labels separate: **verified repair** (contract proved by the permitted procedure and hidden tests pass), **behaviourally correct but unverified**, **verified but behaviourally wrong** (signals a weak contract), **integrity violation**, and **unresolved**.

Measurements

Metric	Role
Verified repair rate	Primary outcome

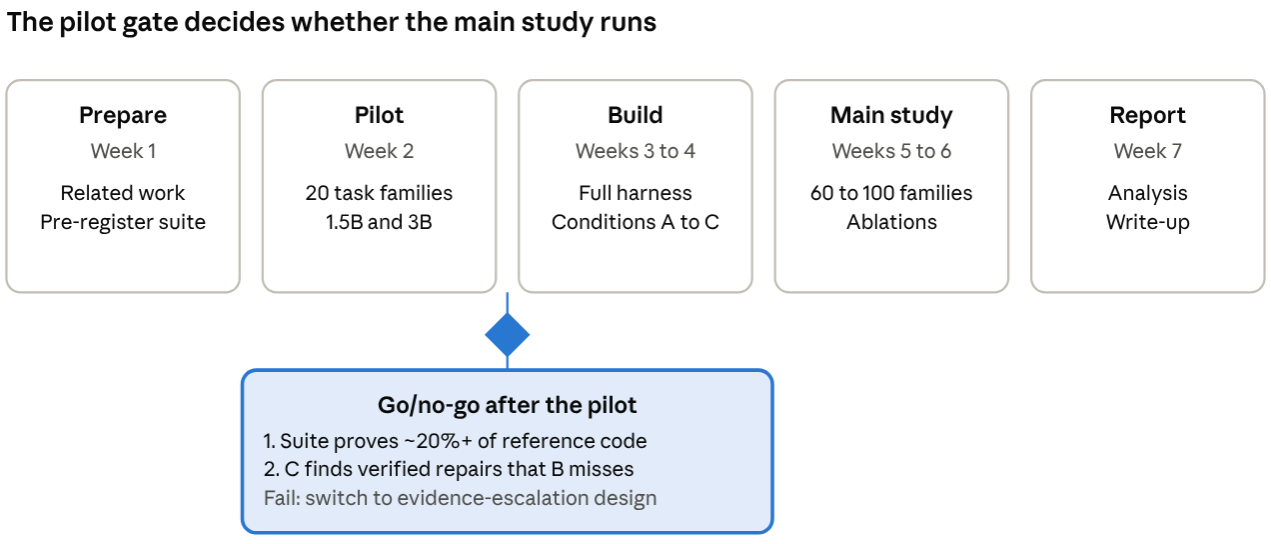
Metric	Role
Share of successes with zero model-written tactics	Mechanism (RQ2)
Model calls and tokens per success	Efficiency
Wall-clock time; Lean CPU time; peak VRAM	Cost
Patch size; regressions on hidden tests	Code quality
Spec-copying and weakening attempts	Integrity (RQ3)
Results by defect type and model size	Where it helps

Analysis

The task family is the unit of analysis. We use paired comparisons across conditions on the same tasks, report effect sizes with bootstrap confidence intervals, and never treat repeated attempts on one task as independent. Pilot tasks used for development are excluded from confirmatory results.

4. Plan of Work

A one-week pilot decides whether the main study runs; if it fails, we switch to a fallback design that reuses the same harness.



plan of work · 5 phases, 1 go/no-go gate

1. **Prepare (week 1).** Read Dissect-and-Restore, APRIL, APOLLO, P³ and the MIT thesis in full, and write a one-paragraph "how we differ" for each. Fix the tactic portfolio, defect types, metrics and analysis plan before any experiment. Set up Lean 4 with cached

dependencies and the local model server.

2. **Pilot (week 2).** Select 20 VERINA task families. Validate references and contracts, inject defects, and run conditions B and C on both models.
   - **Gate 1 (screening):** the portfolio proves roughly 20% or more of the reference implementations without model-written tactics. If not, add the pre-documented induction templates or restrict to the basic tier.
   - **Gate 2 (decisive):** condition C produces verified repairs that B does not. Passing Gate 1 alone is not enough.
   - **Fallback:** if Gate 2 fails, switch to *budget-aware evidence escalation* (minimal versus full versus progressive diagnostic feedback for small models), which uses the same harness and tasks.
3. **Build (weeks 3–4).** Implement all conditions behind one tool interface: compilation, test execution, the tactic portfolio, integrity checks, and full trajectory logging.
4. **Main study (weeks 5–6).** Run 60–100 held-out task families across conditions A–C and both models, with repeated runs where budget allows. Add condition D only if time remains.
5. **Report (week 7).** Paired analysis, failure analysis of representative trajectories, and a paper-ready manuscript. Negative results are reported.

Deliverables: research report, working repair prototype, VERINA-derived repair tasks and evaluation scripts (subject to licences), experiment logs, and the pre-registered portfolio.

5. Threats to Validity

Construct validity. A green proof is not semantic correctness. Benchmark specifications can be wrong: one 2026 audit found issues in 16 of 189 VERINA specifications and 18 of 161 CLEVER specifications [14]. We exclude tasks with known specification defects from the primary analysis and require hidden behavioural tests for every verified repair.

Task-selection validity. Not every VERINA or CLEVER task suits automation-only verification. Picking tasks the portfolio handles well would inflate the method's apparent benefit. We publish the screening criteria and the counts of screened, included and excluded tasks, and limit conclusions to the evaluated task distribution.

Hand-tuning bias. The tactic portfolio could be tuned to the evaluation set. It is pre-registered before the pilot, any change is reported, and pilot tasks never enter confirmatory results.

Specification gaming. The model may copy an executable specification into the implementation, or exploit a weak contract. We protect the contract and tests, flag spec-copying, and report "verified but behaviourally wrong" outcomes separately.

Internal validity. Condition C may benefit from more sampling rather than from code-shaping. All conditions share the same model-call budget, wall-clock cap and preflight; B receives the same automation preflight as C.

Baseline validity. Conditions B and C deliberately differ in capability: B lets the model write proofs, C delegates proofs to fixed automation. That difference is the research question, not a confound. Everything else is shared: tasks, contracts, models, preflight and maximum interaction budget.

Floor effects. Small models may score near zero on proofs, making comparisons uninformative. The pilot gates detect this early; we then narrow to easier tiers or switch to the fallback design.

External validity. VERINA tasks are small functions (median about 9 lines). Results may not transfer to repository-scale Lean projects or other languages. Conclusions are limited to the studied tasks; CLEVER replication is an optional check.

Novelty risk. Code restructuring for verification [8] and transformation-based repair [15] already exist in Dafny. Our claim rests on the Lean 4, automation-only, small-model and repair-setting combination, and on the measured B-versus-C result. We will finalise the novelty statement only after the full related-work comparison.

Statistical validity. A modest number of task families limits per-category conclusions. We report uncertainty, per-category results and negative findings, with the task family as the unit of analysis.

6. Resources and Prerequisites

Everything runs on one laptop; no training and no paid API is required.

Resource	Requirement
GPU	6–8 GB VRAM (1.5B and 3B at 4-bit; 7B optional if measured to fit)
System RAM	16 GB or more (Lean and Mathlib builds are memory-heavy)
CPU	Multiple cores for parallel Lean checking of candidates, the expected bottleneck
Inference	llama.cpp or vLLM with quantized Qwen2.5-Coder models
Verification	Lean 4 with cached dependencies; VERINA evaluation harness

Resource	Requirement
Orchestration	Python scripts for task management, agent loop, logging and statistics
Collaboration	Git and GitHub for code, tasks and logs

Knowledge needed: Lean 4 syntax and its automation tactics, writing and validating contracts, defect injection and testing, local LLM inference, Python orchestration, and paired statistical analysis. The team must also distinguish a wrong implementation, a wrong contract, a failed proof attempt, and an unresolved result.

Target venue: a workshop paper (for example LLM4Code or FormaliSE). ASE NIER or a main-track SE submission only if condition C beats B by a clear, reproducible margin.

7. Preliminary References

1. Z. Ye et al. VERINA: Benchmarking Verifiable Code Generation. ICLR 2026.
2. Learning to Repair Lean Proofs from Compiler Feedback (APRIL). arXiv, 2026.
3. A. Ospanov, F. Farnia, R. Yousefzadeh. APOLLO: Automated LLM and Lean Collaboration for Advanced Formal Reasoning. arXiv, 2025.
4. ImProver: Agent-Based Automated Proof Optimization. ICLR 2025.
5. J. Lu et al. Lean Refactor: Multi-Objective Controllable Proof Optimization via Agentic Strategy Search. ICML 2026.
6. Z. Li et al. P³: Joint Program-and-Proof Planning for Verified Code Generation. arXiv, 2026.
7. Z. Ye et al. Vero: Can AI Agents Build Formally Verified Software Repositories? arXiv, 2026.
8. Dissect-and-Restore: AI-based Code Verification with Transient Refactoring. arXiv, 2025.
9. Verifier environments for LLM generation of verified programs and proofs. MIT thesis, June 2026.
10. D. Dai et al. FeedbackEval: A Benchmark for Evaluating LLMs in Feedback-Driven Code Repair Tasks. arXiv, 2025 (rev. 2026).
11. Effective LLM Code Refinement via Property-Oriented and Structurally Minimal Feedback. arXiv, 2026.
12. T. Guo. Evaluating Symbolic Feedback Granularity for LLM-Based Program Repair. ASE 2026, APR track.
13. A. Thakur et al. CLEVER: A Curated Benchmark for Formally Verified Code Generation. NeurIPS 2025.

14. Certified Program Synthesis with a Multi-Modal Verifier. arXiv, 2026.
15. DafnyFix: Single-Transformation Automated Repair in Dafny. ASE 2026, APR track.

These references mark the closest research directions, not a complete review. The related-work comparison in week 1 will check each against the proposed method before the novelty claim is finalised.